CYBERSECURITY PROJECT REPORT

Phishing URL Risk Checker

Submitted by: Rasika Shinde

Course: Cybersecurity Bootcamp — IBM SkillsBuild

Project type: Beginner-level cybersecurity awareness tool

This report describes an educational program that identifies common warning signs in URLs without visiting them.

1. Abstract

Phishing is a social-engineering technique in which attackers try to trick people into revealing sensitive information or opening harmful links. This project implements a small Python tool that reviews a URL for selected warning signs and displays a LOW, MEDIUM, or HIGH heuristic risk level. It is designed for learning and awareness; it does not visit the URL or prove that a site is safe or malicious.

2. Problem Statement

Users often receive links through email, text messages, and social media. Some links imitate trusted services or use confusing domains. A simple educational checker can help users pause and inspect suspicious features before clicking a link.

3. Objectives

• Build a command-line URL checker using Python.

• Detect selected URL warning signs with understandable rules.

• Display a risk label and reasons for the result.

• Keep the tool offline so submitted URLs are not sent to an external service.

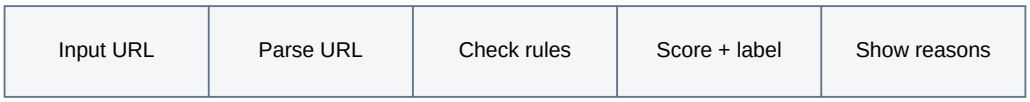
• Explain the limitations of rule-based detection.

4. Tools and Technologies

Component	Purpose
Python 3.9+	Programming language
urllib.parse	Parse URL components
re	Recognize IP-address host patterns
Standard library only	No external package installation required

5. System Design / Workflow

The program accepts a URL as text, adds a default scheme when one is missing, parses the hostname, checks a set of warning signs, adds points for matching rules, maps the score to a risk label, and prints the reasons. No HTTP request is made.



6. Detection Rules

• Non-HTTPS scheme: adds a warning because the connection is not indicated as encrypted.

• IP address as hostname: may be unusual for a public-facing service.

• Long URL or multiple hyphens: can be used in confusing links, but also occur on legitimate sites.

• URL shortener: hides the final destination until expanded.

• Punycode domain: may be used in internationalized domain names and deserves careful inspection.

• Social-engineering words: terms such as login, verify, or password may appear in legitimate URLs too.

• Selected top-level domains: the tool adds a small warning for a configured list; this alone is not evidence of maliciousness.

7. Risk Scoring

The program adds points when rules match. Scores of 0–2 are labeled LOW, 3–4 MEDIUM, and 5 or more HIGH. This score is only a simple teaching aid; it is not a calibrated probability and should not be interpreted as a security verdict.

8. How to Run

Install Python 3.9 or newer. Download phishing_url_checker.py and open a terminal in its folder. Run: `python phishing_url_checker.py`. Enter a URL when prompted, or type q to exit. The requirements.txt file documents that no third-party packages are needed.

9. Testing

Test input	Expected observation
https://example.com	Usually LOW; no listed warning signs
http://192.0.2.10/login/verify	Higher score due to HTTP, IP host, and keyword matches
https://bit.ly/example	Warning for a known URL shortener
blank input	Clear input error message

10. Limitations

A URL may be dangerous even when this tool reports LOW risk, and a legitimate URL may trigger warnings. The program does not examine webpage content, redirects, domain age, certificates, DNS, or threat-intelligence sources. HTTPS only protects the connection; it does not guarantee that the destination is trustworthy.

11. Ethical and Safety Considerations

The tool is passive: it analyzes the text of a URL and does not open it. Users should avoid entering private URLs that contain session identifiers or access tokens. For suspicious messages, do not click the link; navigate to the service through its official app or manually typed website and report the message through the appropriate channel.

12. Future Scope

Possible improvements include a graphical interface, unit tests, a larger set of transparent rules, an optional reputation service with user consent, and evaluation on a labeled dataset. Any external

service integration should protect user privacy and clearly document its data handling.

13. Conclusion

The Phishing URL Risk Checker demonstrates basic URL parsing, rule-based scoring, and security awareness using Python. Its main value is explaining warning signs in a transparent way. It should be used as a learning project and not as a replacement for security products or careful user judgment.

14. References

1. IBM SkillsBuild — Cybersecurity learning resources (course context).

2. Python documentation — urllib.parse: https://docs.python.org/3/library/urllib.parse.html

3. CISA — Avoiding Social Engineering and Phishing Attacks: https://www.cisa.gov/secure-our-world/recognize-and-report-phishing

Prepared as a student project template. Please review it, run the code, and adapt it to your course rubric before submission.